

Computational verification and reviewer guide

Companion to the order-28 Murty–Simon candidate

Paul Lenz*

7 September 2026

Reviewer edition 1; hardened checking release

Abstract

This companion specifies the immutable evidence, checking procedure and review boundaries for the direct order-28 candidate proof. It separates preservation checks, exact computational exclusion, mathematical validity and external review. The reviewer entry point checks the six original component archives, enforces a mandatory guarded version of the v8 residual enumerator in a derivative working copy, replays both edge-count scopes, and checks five stage handoffs. The original sources, evidence, timing reports and manifests are not rewritten. The new entry point performs full checking by default; integrity-only operation must be explicitly requested. This is a reproducibility and review document, not an independent endorsement of the theorem.

1 What is being reviewed

The companion manuscript, *The Murty–Simon bound at order 28: a computer-assisted candidate proof*, proposes

$$e(G) \leq 196, \quad e(G) = 196 \iff G \cong K_{14,14}$$

for simple diameter-two edge-critical graphs of order 28. Infinite distance is allowed when deletion disconnects a graph. The direct route has two separate degree-15 calculations: $(a, b, t) = (12, 15, 1)$ at 196 edges and $(12, 15, 2)$ at 197 edges. Other maximum degrees and the regular equality boundary have explicit arguments. Fan’s cited strict bound of 197.2075 is the external input excluding 198 or more edges.

The direct route uses **LocalIncidence-v7**, whose final equality partition is $22 + 19 + 347$. It does not use the separate **degree-load-v7** route, whose partition is $173 + 215$ and which adds a weak-core density reduction. The latter’s numerical bundle and that reduction are not required by this release. Alternative exclusions of the same row are not additional rows.

The frozen $n=25$ and $n=27$ candidates are separate papers. This release neither changes those editions nor promotes the governed theorem ledger. It creates an editorial and engineering layer over the direct $n=28$ evidence.

2 Deliverables and immutable baseline

The repository is `paullenz/MurtySimon25`. The baseline inspected for this release is commit

*Project direction: Paul Lenz. Software, manuscript drafting and internal checking: ChatGPT (“Geeps”). Independent mathematical review and external reproduction remain open.

cd3c22411911280624a8223035e6de2343abbb7c.

The original proof ZIPs v6, LocalIncidence-v7 and v8 were placed in their dated research directories by commit d72fe669c906bed3ffeb8847913e13d49f7a3ef7. The internal audit payload was published in 6176ce6428c105b791d9129682cbacd55137edd1, followed by its receipt at the baseline above. These are publication identifiers, not proof certificates.

The new release contains the main manuscript and this companion in readable and typesetting-source formats; the guarded helper; its exact patch and provenance; the default review driver; malformed-input and full-domain regression tests; archive and file hash inventories; a review report template; and separate records of the release validation. Original archives remain byte-identical dependencies. An optional all-in-one download includes copies of those exact archives for offline replay; a repository checkout may recover them through the older checkpoint guides.

2.1 Original archive inventory

The following table gives the original dependencies. Each archive contains one top-level directory with the corresponding base name.

Archive base name	Bytes	Payloads
MurtySimon_GeneralN_DemandSupport_v3	1,258,114	41
MurtySimon_GeneralN_LabelTail_v4	566,681	40
MurtySimon_GeneralN_ColumnPropagation_v5	1,035,233	54
MurtySimon_GeneralN_SharedAdjacency_v6	17,940,219	88
MurtySimon_GeneralN_LocalIncidence_v7	15,943,487	87
MurtySimon_N28_197_Direct_v8	5,677,524	90
Total original payloads, excluding six manifests		400

The complete SHA-256 values follow; the machine-readable source is `ARCHIVES.json`.

v3: 4d5e06a85f2e9112776205a632436a8e9224
bc9f14055ceabf5a67dd83d89fd1

v4: 3944e786fb0ea5322ae3e18dd177ef8b0104
6d903076a4ca61caf89fdc5ae184

v5: 79bdbdf02b23b9eef38e964b54eb300921b2
4d539014f07b476655d1575e63b6

v6: d00a9627e9ed4d8a6f22ae1f2b194791412e
e2d16d502b1968be16e02aa30496

v7: 0637c6b91d7e0c311025bf68dc7a373f57f6
230e271588bd6693d8ffc17eb6cf

v8: b753076ffc755066a0c57756be91cc5b265c
163d869244d3aa65e3943799347b

The two displayed segments of each hash are concatenated without whitespace. These are archive hashes, not hashes of an arbitrary recompressed replacement. The release driver also pins the original manifest hashes and verifies every original payload's length and SHA-256 after safe extraction.

3 Run the hardened checking release

Use Python 3.10 or later with assertions enabled, a C++17 compiler invoked as `g++`, and Boost headers. No optimisation solver or network connection is used during checking. Installation of prerequisites is separate from the checking run. The original capsule recovery programs

for v3/v4 were tested in CPython 3.13.5 and fail closed on incompatible byte reconstruction; obtaining the exact original ZIPs avoids that extra recovery dependency.

Place the six original ZIPs in one directory. From the reviewer release directory run:

```
python3 -I -B review_check.py \
  --archives-dir /absolute/path/to/original-zips \
  --output /absolute/path/to/new-review-run --jobs 3
```

This command performs full checking by default. The output must not exist and must be outside the release directory. Every component writes to a new working copy or separate result directory. At most the requested number of component subprocesses run concurrently within this invocation. A failure produces a nonzero exit code and preserves the corresponding logs; it must not be treated as an exclusion.

For an explicit integrity-only pass:

```
python3 -I -B review_check.py \
  --archives-dir /absolute/path/to/original-zips \
  --output /absolute/path/to/new-integrity-run --verify-only
```

That result says `arithmetic_replayed: false`. It does not certify either density scope. Do not use `-0`, a smoke-test result, or a standalone helper's successful exit code as a substitute for the full command.

3.1 What the driver does

Before numerical work it verifies archive sizes and hashes, CRCs, safe member paths, duplicate membership, exact manifest coverage, all 400 payload lengths and hashes, and original manifest identities. The v8 helper is replaced only in a derivative working copy, after its original hash and the hardened file hash match the pinned overlay record. The original manifest is retained for provenance; the derivative is not represented as an unchanged original package.

The driver executes the complete v3 checker; v4 in full `-replay` mode; the complete v5, v6 and LocalIncidence-v7 checking wrappers; the complete v8 checker with the guarded helper; all-degree scope arithmetic; and the new helper regressions. It compares the fresh v8 report to the preserved result, ignoring only declared timing fields. It compares all five original equality handoffs below and rechecks the immutable archives and extracted payloads after execution.

Boundary	Exact data comparison
v3 to v4	v3 <code>exploration/extra_n28_b15.json</code> against v4's saved v3 demand frontier
v4 to v5	Complete retained demand list
v4 to v5	Complete projected residual-row list
v5 to v6	Final 6,918 rows, with column options, caps and forced labels
v6 to v7	Final 388 rows and their retained data

Compressed copies are decompressed before the documented byte comparisons. These are five interfaces, not five independent proofs. The 197-edge checker generates and validates its own complete $t = 2$ domain and internal handoffs.

3.2 Success output and its limits

A successful full run writes `REVIEW_CHECK_REPORT.json`, `APPLIED_OVERLAY.json`, named command/exit-code logs, component replay directories, the v8 exact report, and the helper-regression report. The summary records full arithmetic replay, the mandatory hardened helper,

zero final survivors in both degree-15 density scopes, five exact handoffs, and unchanged original payloads.

The driver does not re-prove graph-theoretic lemmas, re-audit Fan’s original article, formalise the argument in a proof assistant, or certify the alternative degree-load-v7 route. A reviewer must assess those mathematical premises separately. The full original v6/v7 component wrappers also replay their saved graph-lift and abstract-system regressions, whose sampling limitations remain unchanged.

4 The input-validation correction

4.1 The defect and why it is non-blocking for the original driver

The committed internal audit’s RT-01 finding concerns the standalone v8 `src/check_rows.cpp`. It accepted an unsupported residual interval such as 64..64 and could produce a successful empty report. Such an output is not an exclusion certificate.

The original `check197.py` reconstructs the exact initial domain and verifies $15 \leq r_{\min} \leq r_{\max} \leq 63$ for every retained interval. It verifies the complete demand-input file before invoking the helper. Thus the audit’s malformed input does not reach the helper through the admitted proof entry point. The recorded valid-domain calculation is not invalidated by that standalone weakness.

4.2 The reviewer-release correction

This release implements the documented guard correction as a new, versioned derivative, not as a silent edit of historical evidence. It does not claim that this new guard source is byte-identical to the earlier audit patch. It preloads and validates the entire helper input before opening outputs: positive bounded record count; all twelve demands in range; sorted demands; supported ordered intervals; strictly ordered, non-duplicate demand records; complete records; and no trailing data. It also rejects identical path strings and checks output write failures. These are operational input guards, not a claim of comprehensive security hardening against every filesystem adversary.

The enumeration, capacity refinement, arithmetic comparisons, row digest, survivor serialisation and per-demand band loops are unchanged. `OVERLAY.json` pins both source versions, and `check_rows_input_guard.patch` shows the entire difference.

The release tests exercise 13 malformed cases: out-of-range and inverted intervals; negative, zero, excessive and unrepresentable counts; truncated input; negative and oversized demands; unsorted demands; duplicate records; and trailing data. Every such test must return nonzero before output creation. The valid-domain regression must match the original complete 8,216,928-row report, all survivor bytes and all per-demand band bytes.

The full reviewer driver applies the overlay automatically. A reviewer should not have to remember an optional patch step. Original source and original archives remain available for exact historical reproduction.

5 Expected exact outcomes

The manuscript supplies the mathematical definitions. The principal numeric checkpoints are:

Quantity	196-edge scope	197-edge scope
Charging demand tuples	18,645	12,012
Retained demand intervals	1,976	1,229
Residual rows enumerated	17,669,896	8,216,928
After row screen	24,411	5,154
After projected tests	13,196	2,959
After one-source joint projection	7,725	1,584
After v5 activation	6,918	Not a separate stage
After shared model	4,617	797
After source-degree model	479	7
After complete v6 branch trees	388	No branching used
After LocalIncidence/endpoint completion	0	0

The equality endpoint completion has $22 + 19 + 347 = 388$ exact rejections. The 197-edge completion has $787 + 790 + 7 = 1,584$ exact rejections. The separate degree-16 equality scope has 39 initial demand tuples and 604 final residual rows, all excluded. Those units must not be added together as graph counts.

The audit's 8,983 final certificate leaves comprise 2,301 shared-model leaves, 4,138 source-type leaves, 572 leaves belonging to 91 complete equality branching trees, 388 LocalIncidence-v7 leaves and 1,584 direct197-v8 leaves. A leaf count is not a count of distinct initial rows. The certificate and branch checkers reject incomplete or duplicated disposition lists.

6 Mathematical review versus numerical reproduction

6.1 The key logical implication

Every hypothetical critical graph in the claimed scope must yield one of the enumerated records, and must lift to a feasible point of the necessary system attached to that record. Exact arithmetic only establishes that the system is impossible. An erroneous universal constraint could invalidate the graph-to-system implication even while all arithmetic certificates remain valid. This is the central reason for independent mathematical review.

The graph arguments needing particular attention are the uniqueness of quasi-edge exceptions; disjoint residual injections and their empty-set cases; the strict positive-surplus premise of residual activity; exclusion of isolated $H[A]$ vertices; the distinction $x_i \geq s_i$ rather than $x_i = s_i$; and source/supplement pair uniqueness across orientations. The endpoint bound $R_i + x_i \geq q_u + p_u$ must concern the same incidence and actual degree types.

For the finite reductions, inspect domain membership and independent counts; paired column options and exact slack; the preservation of actual witnesses under repeated matching and cap refinement; activation-cost normalisation by incoming capacities; and source-conditioned fixed totals. Symmetry classes are an averaging device, not assumed automorphisms. Distinct-endpoint corrections and class-size factors must be correct. Branches must split genuine integral graph counts, not raw fractional averages.

The source-local residual injection uses quasi-edges for missing pairs inside A . It is validly applied to the original critical graph in the direct route. It must not be imported without proof into a weaker object retaining only selected B -quasi-edges. That distinction separates this proof from the alternative weak-core route.

6.2 Independence and regression limits

The original discovery and checking programs were separately implemented, but by the same assistant. The additional audit arithmetic kernel uses a fresh integer accumulator and branch

checker while reusing preserved model constructors. It is not a third independent derivation of the graph constraints.

The committed audit reports fresh tests on 107 labelled critical graphs and no tested key-lemma failure. Its sample has no positive-surplus graph. The original component tests also include actual-graph lifts, fractional symmetry averages and abstract incidence systems. None of these finite samples proves a universal structural lemma or supplies empirical evidence that all hypothetical dense graphs have been observed. Their purpose is to catch implementation and local-argument errors.

No blocking defect was found in the audited direct route. That is a recorded internal audit conclusion, not a guarantee that no defect exists. No independent referee endorsement, publication acceptance, novelty/priority determination, improved general coefficient or formal-kernel verification is asserted.

7 Release validation executed

The new default entry point was executed in full in a fresh directory on 7 September 2026. All eight recorded jobs passed: the six component checking routes, the all-degree scope calculation, and the hardened-helper regressions. Both edge-count scopes end with zero survivors. All five handoffs match byte-for-byte; all six original ZIPs and all 400 original extracted payloads remain unchanged. The hardened copy rejects all thirteen malformed-input controls and reproduces the original valid-domain report, survivor bytes and per-demand band bytes.

These are tests of this new reviewer entry point, not merely a quotation of the earlier audit. The complete report is `validation/REVIEW_CHECK_REPORT.json`; its commands, return codes and timings describe this run. Earlier wrapper-development path mistakes were corrected before that successful execution and are identified separately in the release provenance. This internal execution is not independent external reproduction. The mathematical review status remains open.

8 How a reviewer can report a result

The release includes `REVIEW_REPORT_TEMPLATE.md`. A computational reviewer should identify the exact repository commit, archive hashes, operating system, Python and compiler versions, command, exit code, fresh report hashes, and any deviation from the expected output. A successful local run should not be described as proof of the universal lemmas.

A mathematical reviewer should name the precise lemma, equation, domain reduction or lifting rule examined; record whether it is verified, requires clarification, or has a counterexample; and distinguish a presentation improvement from a substantive gap. For a proposed defect, a concrete graph, small abstract system, missing case, or invalid inference is more informative than a general confidence score. For a code issue, preserve minimal input, stderr, exact command and the expected versus observed behaviour.

External review statuses start as `OPEN`. They must be changed only on receipt of an actual assessment, not merely because an invitation was sent or an internal replay passed. The $n=25/n=27$ frozen editions and original $n=28$ archives are not silently rewritten in response to editorial corrections; later editions record their differences.

9 References and source map

The companion manuscript contains the external bibliography, including Fan (1987), Wang (2012), and Haynes–Henning–van der Merwe–Yeo (2014). The main external density statement used is Wang’s report of Fan’s strict bound on page 2 of arXiv:1205.4397.

The exact computational sources in the original archives are as follows. Paths are relative to each archive's extracted root.

Component	Primary review entry points
v3	PROOF.md, check.py, evidence/n28_delta16.json.gz
v4	PROOF.md, replay.py, check_rows.cpp, initial and projected evidence
v5	PROOF.md, replay.py, the separate joint and activation checkers
v6	PROOF.md, replay.py, named-constraint and complete branching checkers
Local v7	PROOF.md, src/check.py, src/check_local.py, src/check_ledger.py, src/check_total.py
v8	PROOF.md, src/check197.py, src/check_scope197.py, original src/check_rows.cpp
Internal audit	Repository <code>project/reviews/n28/2026-09-07-redteam-v1/</code> : REPORT.md, RESULTS.json, recover_audit.py and storage manifest

The release's original-archive inventory is authoritative for evidence identity. The source map tells a reviewer where to begin; it is not a substitute for recovering the full referenced payloads. The current release validation and the original audit are separately dated and reported.